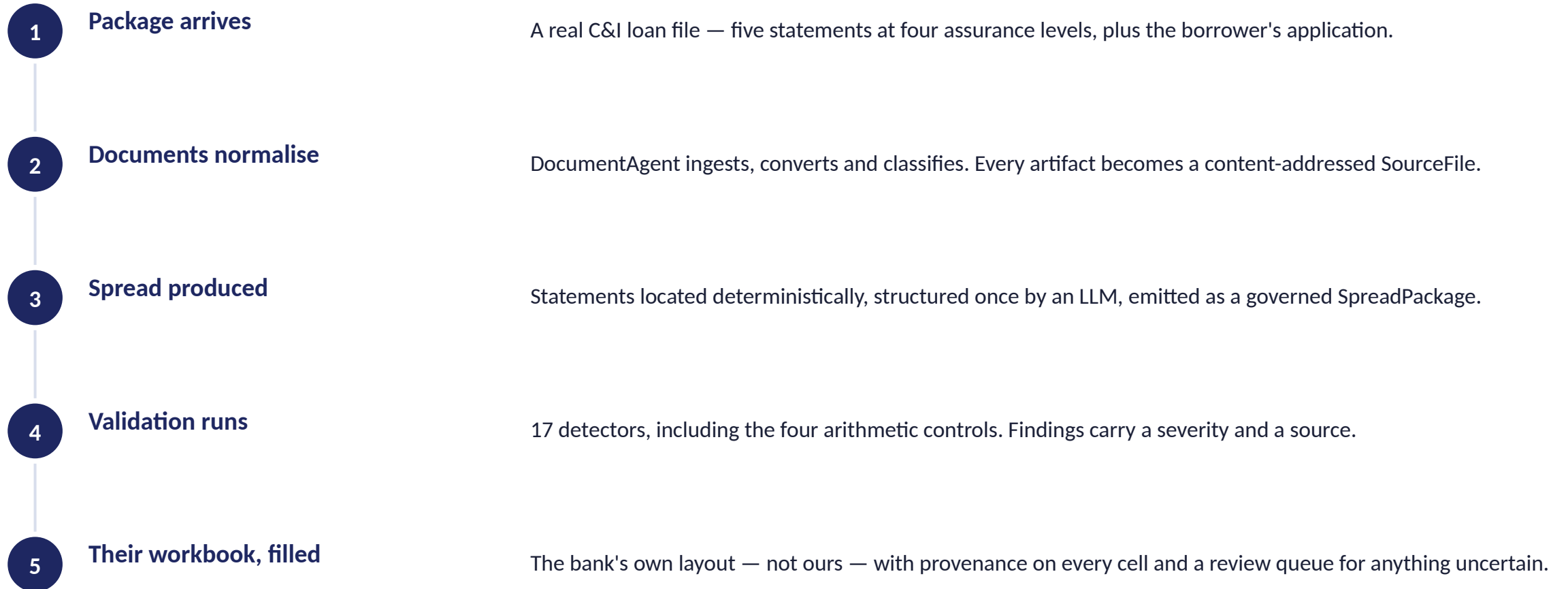


VP-2 • FINANCIAL SPREADING

The customer's own spread, produced by the platform

How an FDE turns a bank's existing workbook into a running capability — and how heavily VP-2 leans on the JazzX SDK.

What you'll see tomorrow



Nobody touches the pipeline between step 1 and step 5. The human work happened once, at onboarding.

Four moments. Only two involve a human.

Once

Onboard

CUSTOMER

Hands over the spread workbook they already use, plus one sample package.

Once, hours

Induce

FDE + AGENT

The workbook is parsed into an output template. The FDE reviews and publishes.

Once, plus as cases surface

Teach

FDE

Writes skills for what the template cannot express.

Every transaction

Run

NOBODY

Document agent → fill agent → validation → governed workbook.

The measure of success is not the first customer. It is that the tenth is boring to build.

A spread is an institution's opinion about which lines matter. Two banks reading the same filing produce different spreads, and both are right — so the template is a contract, not a layout.

Case 1 — Regional Bank (C&I)

Their EBITDA is not EBITDA

CIT EBITDA = Gross Profit – SG&A – Rent

Never touches operating income, D&A or interest — and it subtracts rent. It is what their covenants reference, and it exists only in their spreadsheet.

Adj. EBITDAR is a trap. Same word as the pack's existing metric, different lineage, different number. Binding it to the pack metric is a silent, plausible, wrong answer.

Four rows are labelled “Margin”. Label is not identity — every binding resolves by key.

- **The LTM rule differs by line semantics**

Flow rows roll forward, stock rows take the latest interim. Verified: $61,443 - 44,326 + 57,581 = 74,698$, matching their cell exactly.

- **Two hardcoded plugs, in their reference file**

An analyst typed `=10853-1` and `=25007-D37` into data cells. We record them as defects rather than reproduce them.

- **Assumptions dressed as figures**

Maintenance capex is a flat 1,000 every period — a convention, not a measurement. It must render as an assumption.

- **Cash interest ≠ interest expense**

1,892 vs 2,096. Collapsing them corrupts the fixed-charge coverage ratio, which is a covenant.

Case 2 — Mesa Verde (CRE multifamily)

+38.0%

Insurance, T-12 vs Year 2

193,200 → 266,600

+6.2%

Total operating expenses

every other line moves 2–3%

1.30x

DSCR, T-12 — still pre-flag

NOI 3,559,000 / DS 2,740,000

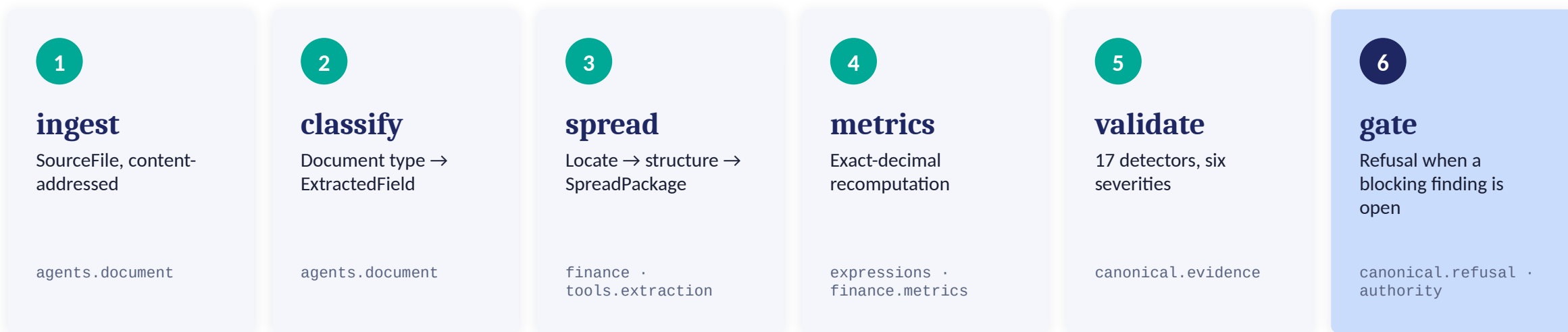
“Note: Insurance and Property Tax pending analyst confirmation. Post-flag NOI/DSCR locks once cleared.”

Row 19 of their sheet, verbatim — and rows 15 and 17 are literally labelled “NOI (pre-flag)” and “DSCR (pre-flag)”. The customer put workflow state in a free-text note because a spreadsheet has nowhere else to put it.

This is the clearest argument for why the template is not the whole contract.

No schema of rows and formats can express “these two figures are provisional until a human clears two specific inputs, and while provisional they must not reach the credit decision.” That is a skill — and it becomes a gate with candidate-versus-approved discipline, promoted by a human with an actor recorded.

Every stage is an SDK primitive



The whole pipeline is data, not code.

Six steps declared in `pipelines.yaml`, resolved through the SDK's `StepRegistry` and run by `ConductorEngine`. A new segment reorders or swaps steps without touching Python. The lifecycle itself is an SDK `StateMachine` — illegal transitions return a typed `Refusal`, not an exception.

How heavily VP-2 leans on the SDK

67

distinct SDK modules

imported on the spread path

153

import statements

73%

of files on the path

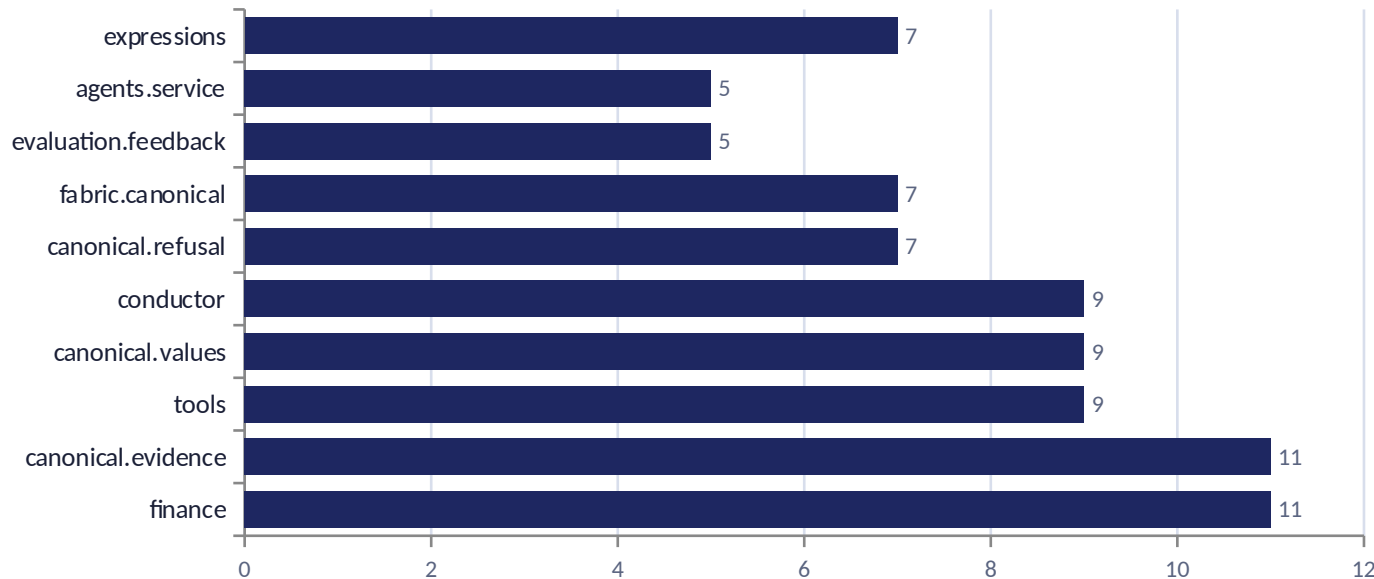
43 of 59 import jazzx_sdk

13,162

lines on the spread path

capabilities/ + ci_spread/

Top SDK subsystems on the spread path (import sites)



What that buys

- Provenance and confidence on every cell — the XF-1 gate is an SDK assertion, not our convention.
- Exact-decimal metrics through the SDK expression DSL, statically validated at pack-load.
- Authority, refusals and maker-checker from fabric.canonical — governance we did not write.
- Excel export, workbook layout and line vocabulary all delegated to jazzx_sdk.finance.

Three layers, and the moat is the middle one

SDK

jazzx_sdk — horizontal

- Canonical chain: Policy, Evidence, Decision, Trace, Outcome
- Conductor, StepRegistry, StateMachine
- finance: spread schema, vocabulary, workbook, Excel
- expressions DSL · authority · evaluation · agents

Reusable across every domain. Nothing lending-specific.

PACK

config/packs — the moat

- Chart of accounts · metric catalog · add-backs
- Reclassifications · break-outs · spread templates
- Policies, playbooks, workbook layouts
- Per-customer skill overlays (tier 3)

Authored, versioned, certifiable. This is what compounds.

JACI

src/jaci — thin

- Capability manifests + step registration
- Domain glue and the demo surfaces
- Validation detectors specific to lending
- Shrinking as SDK primitives land

Should keep getting smaller, not bigger.

Governed by construction, not by review

1

Provenance on every cell

Each value carries a SourceCoordinate back to the document region and a content hash. XF-1 asserts it as a test, not a sample.

2

Confidence that means something

Derived from how the value was obtained — exact key, alias, label, substring — so the review queue can rank on it.

3

Validation before promotion

17 detectors across six severities, including the four arithmetic controls now live in the governed pipeline.

4

Human decisions are recorded

Maker-checker over the authority matrix, append-only override lineage, and promotion that never happens on a re-run.

None of this is spreading-specific. It is the canonical chain doing its job — which is why the next domain inherits it for free.

What is real, what is staged, what is not built

REAL AND RUNNING

- Governed spread pipeline end-to-end on a real 10-K
- SpreadPackage with provenance on 100% of line items
- 17 validation detectors; four arithmetic controls now wired
- Governed workbook export, review queue, maker-checker
- RB reference case reproduces the analyst's LTM column exactly

STAGED FOR THE DEMO

- Acts 2 and 3 of the demo arc are held pending fixtures
- Ground truth is 4 keys plus a 30-row as-filed table
- Per-cell confidence is currently a constant

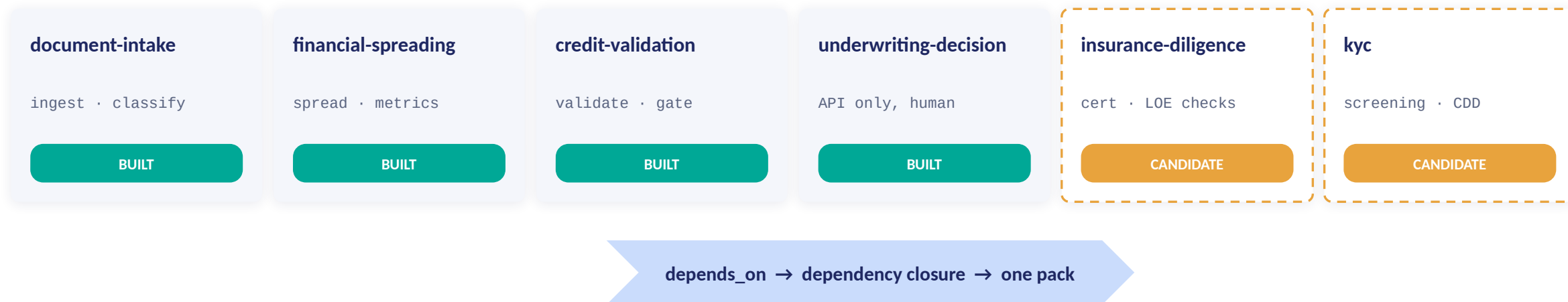
NOT BUILT YET

- OutputTemplate and TemplateFillAgent
- Induction from a customer workbook (CLI verb)
- Skill.applies_to on the SDK Skill model
- Row-by-row spreading accuracy in the eval harness

WHERE THIS GOES

A pack is assembled from capabilities

Spread is already a capability, not a scenario. That is the right shape — and it is how the next pack gets built.



Answer to “should spread be its own scenario?”

Yes as a capability — which it already is — and no as a KYC-style cross-scenario import. Capability manifests with `depends_on` already resolve a dependency closure; that is the mechanism for crafting a new pack.

What is actually missing for a cleaner demo

A spread-only demo surface. The `ci_spread` demo page is a monolith carrying credit analysis and three acts. Splitting the surface is thin work; splitting the capability is already done.

Next

Promote insurance and KYC to capability manifests; build `OutputTemplate` + fill agent; deliver Wave 4b induction through it.

The bank gets its own spread back.

Not our format with their labels on it — their workbook, their house metrics, their gates, filled by the platform with provenance on every cell.

67 SDK modules · 73% of the spread path · six declarative pipeline steps · one capability that composes into the next pack